



NowFootball

Javier Saravia Ogazon



Índice

[Introducción](#)

[Parte A: Análisis del desarrollo móvil](#)

[Parte B: Especificación de la app](#)

[Casos de uso](#)

[Requisitos funcionales](#)

[Requisitos no funcionales](#)

[Parte C: Entornos de desarrollo](#)

[Dispositivos virtuales](#)

[Estructura del proyecto](#)

[Parte D: Recursos de la aplicación](#)

[Textos](#)

[Colores](#)

[Conclusion personal](#)



Introducción

He creado la app NowFootball, una aplicación para ver resultados de fútbol, he elegido este tema porque es una app que yo mismo usaría, cuando llega el finde de semana quiero ver como van los partidos sin tener que abrir una web llena de anuncios. El proyecto esta hecho en Java con vistas XML a partir de la plantilla Empty Views Activity.

El orden de la memoria es:

- Parte A: análisis del desarrollo móvil
- Parte B: especificación de la app
- Parte C: entorno de desarrollo
- Parte D: recursos
- Conclusión personal



Parte A: Análisis del desarrollo móvil

Limitación	Cómo afecta a NowFootball
Batería: la red, la pantalla y la CPU gastan una batería limitada	No puedo pedir resultados cada pocos segundos. Solo actualizaré cuando haya partidos en directo
Conectividad: la conexión es inestable (wifi, 4G, sin cobertura) y los datos son limitados	En un estadio lleno la red va fatal. La app debe mostrar el último resultado guardado con Actualizado hace x min en vez de quedarse en blanco.
Pantalla: es pequeña y se usa con el dedo	En la pantalla se incluye lo necesario (equipos, marcador, y estado).
Memoria: hay poca RAM y Android cierra procesos en segundo plano	Imágenes ligeras (vectores) y un estado que se pueda recuperar si el sistema cierra la app
Fragmentación: miles de modelos, versiones y densidades	Uso dp/sp, recursos por densidad (mipmap-*), icono adaptativo y API 24 para cubrir la mayoría de dispositivos

Parte B: Especificación de la app

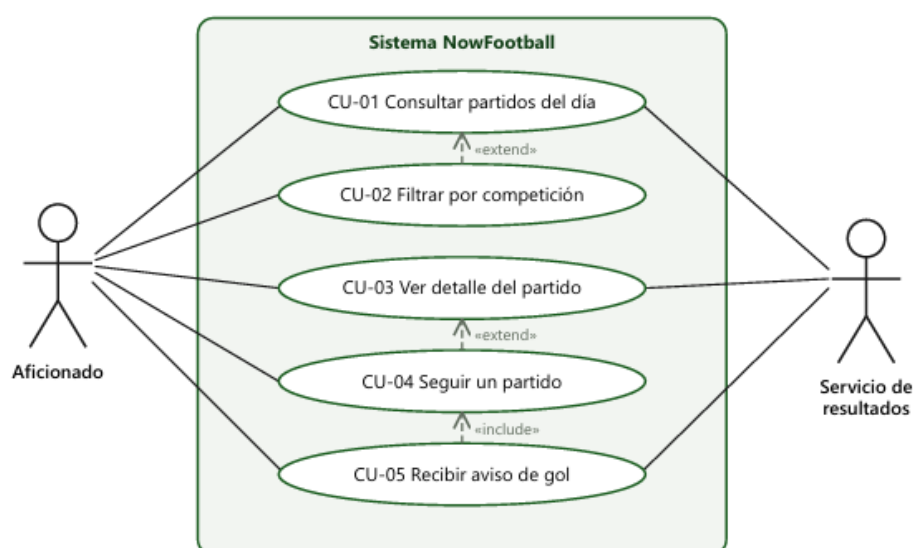
NowFootball es una app para consultar de un vistazo los resultados de fútbol del día: qué partidos se juegan, cuáles están en directo y en que minuto van, cuáles han terminado y qué hora empiezan los siguientes.

Está pensada para aficionados de 16 a 50 años que siguen el fútbol europeo y están interesados en saber los resultados de fútbol.

Objetivos:

- Rapidez: que el usuario pueda ver el resultado de cualquier partido en menos de 5 segundos
- Sencillez: llegar al detalle de cualquier partido con un máximo de 2 toques en la pantalla
- Alcance: que la app esté disponible también en inglés y que funcione en la mayoría de dispositivos móviles.

Casos de uso





Requisitos funcionales

Código	Descripción	Prioridad
1	La app mostrará la lista de partidos del día con equipo local, equipo visitante, marcador y estado	Alta
2	El usuario podrá filtrar los partidos por competición mediante un selector	Alta
3	Al pulsar un partido o Ver detalle, se mostrará el marcador, el estadio y la lista de momentos clave del partido	Alta
4	El usuario podrá marcar un partido como seguido	Media
5	La app enviará una notificación cuando haya un gol en un partido seguido	Media
6	La interfaz se mostrará en el idioma del dispositivo	Media
7	La pantalla principal indicará cuánto tiempo ha pasado desde la última actualización de los datos	Baja



Requisitos no funcionales

Código	Tipo	Descripción
1	Usabilidad	Todo elemento pulsable mide al menos 48 × 48 dp, y se llega al detalle de un partido en menos de 2 toques desde la pantalla principal
2	Compatibilidad	La app se instala y funciona desde Android 7.0 (API 24) hasta Android 16 (API 36). Elijo API 24 porque, según el asistente de Android Studio, cubre aproximadamente el 97 % de los dispositivos activos
3	Rendimiento	La pantalla principal se muestra en menos de 2 s en arranque en frío en el emulador Pixel 7
4	Tamaño	El APK de depuración ocupa menos de 10 MB
5	Accesibilidad	Todas las imágenes tienen contentDescription, los textos usan sp y el contraste texto/fondo es $\geq 4,5:1$
6	Mantenibilidad	Hay 0 textos escritos a mano en los layouts



Parte C: Entornos de desarrollo

He creado el proyecto en Andorid Studio: New Project>Empty Views Activity y esta configuración:

Opción	Valor
Nombre	PMDM_P1_JavierSaravia
Paquete	es.medac.javier.app
Lenguaje	Java, con vistas en XML
API mínima	minSdk 24 / targetSdk 36
Sist. de compilación	Gradle 9.5.0 con kotlin DSL, Andorid Gradle Plugin 9.3.3 y catálogo de versiones
	Git con el .gitignore de Android Studio (excluye build/, .gradle/ y local.properties)

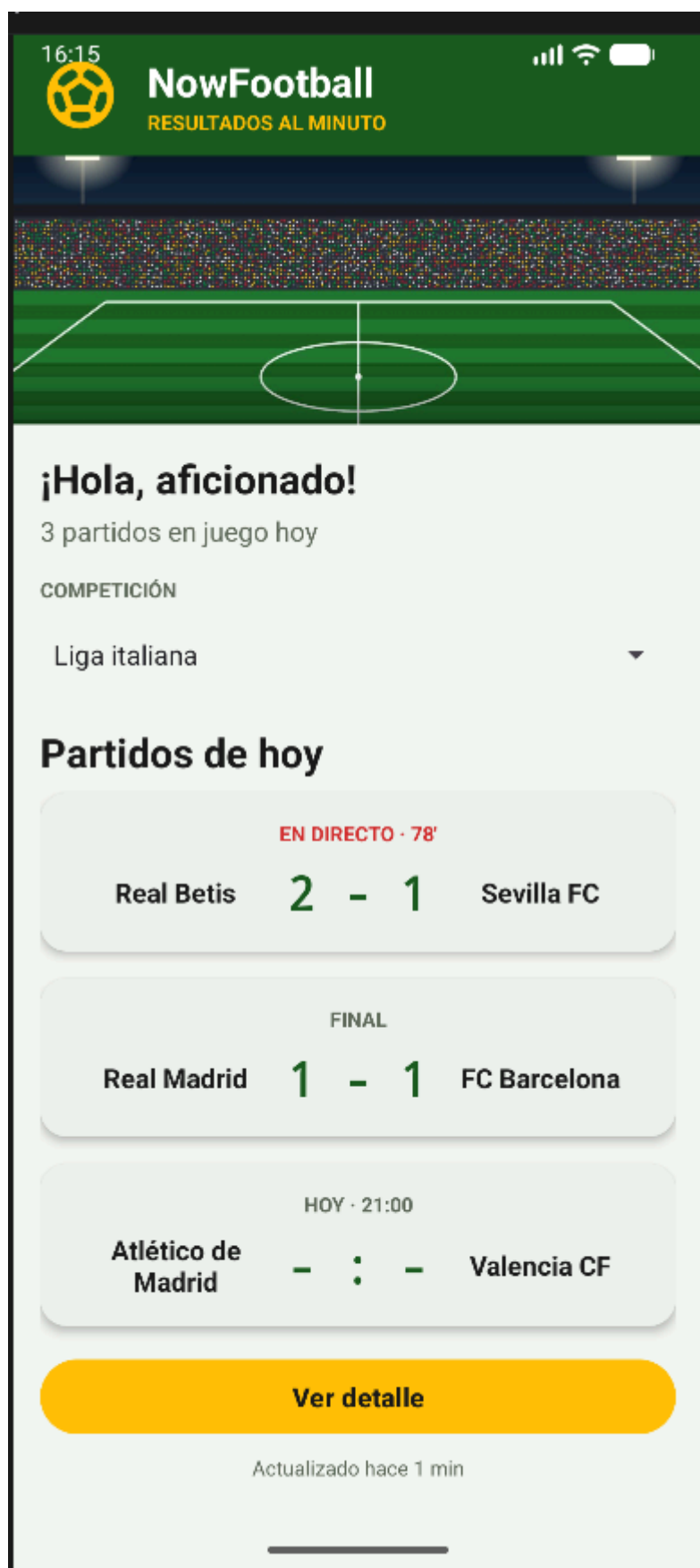
Dispositivos virtuales

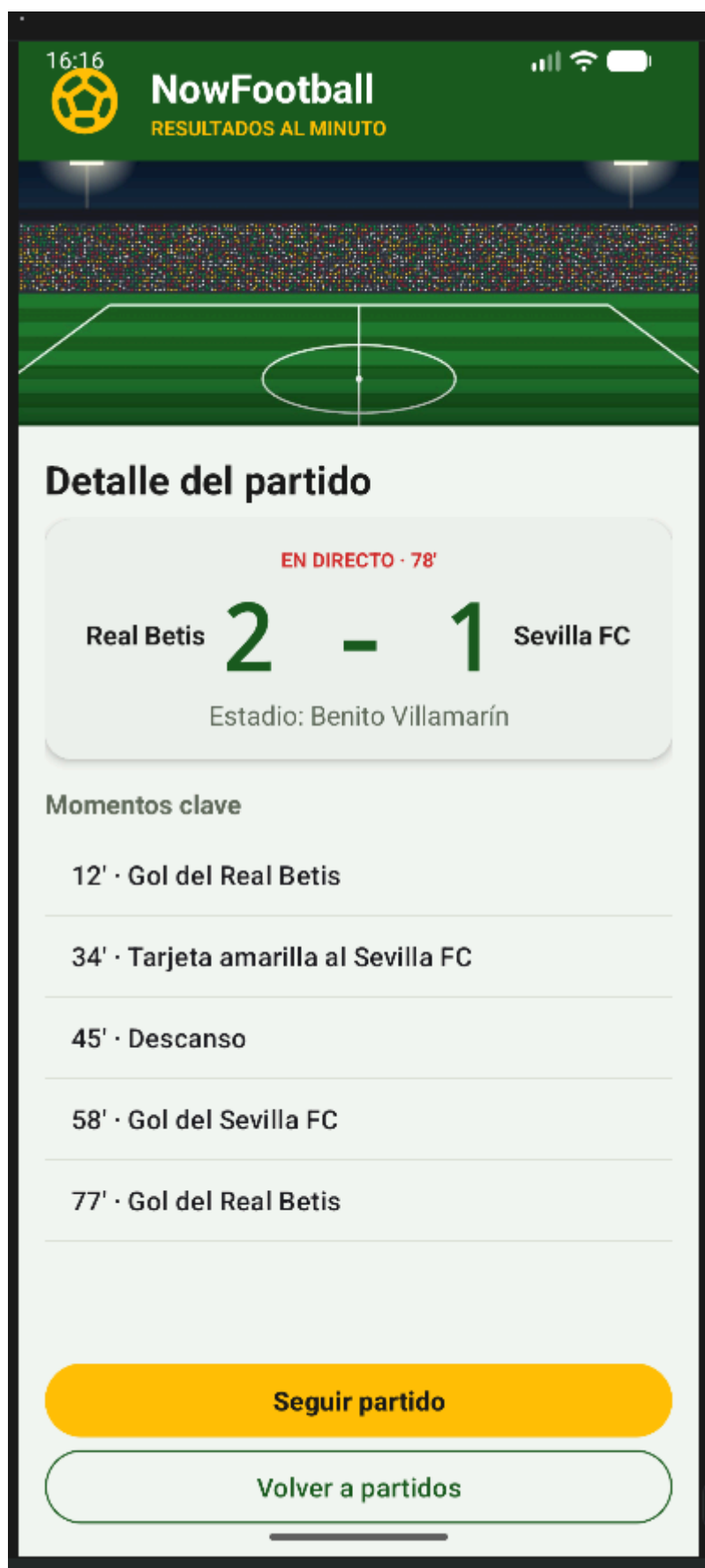
AVD	Tipo	Resolución	Densidad	Versión / API
Pixel 7	Móvil 6,3"	1080 x 2400 px	420dpi	Android 16 / 36
Pixel Tablet	Tablet 10,95"	2560 x 1600 px	320dpi	Android 15 / API 35



Estructura del proyecto

```
PMDM_P1_JavierSaravia/  
├─ settings.gradle.kts      ← nombre del proyecto, módulos y repositorios  
├─ build.gradle.kts        ← configuración común (plugins)  
├─ gradle/libs.versions.toml ← catálogo de versiones de dependencias  
├─ gradle/wrapper/  
└─ app/  
    ├─ build.gradle.kts      ← namespace, applicationId, minSdk, targetSdk, dependencias  
    └─ src/main/  
        ├─ AndroidManifest.xml  
        ├─ java/es/medac/javier/app/  
        │   ├─ MainActivity.java  
        │   └─ DetalleActivity.java  
        └─ res/  
            ├─ layout/ · layout-land/      ← interfaces en XML  
            ├─ values/ · values-en/ · values-night/  
            ├─ drawable/ · drawable-nodpi/ ← vector y bitmap  
            └─ mipmap-*/                  ← icono por densidad
```







Parte D: Recursos de la aplicación

La app tiene dos pantallas: Partidos de hoy (activity_main.xml) y Detalle del partido (activity_detalle.xml).. En los layouts no hay ningún texto, color ni tamaño escrito a mano: todo viene de res/values

Textos

values/strings.xml tiene 28 cadenas, 2 strings-array y 1 plurals

values/strings.xml tiene 28 cadenas, 2 string-array y 1 plurals, ordenados por pantalla y con comentarios.

- string-array: competiciones rellena el Spinner directamente desde el XML, y eventos_partido rellena el ListView del detalle.
- Textos con parámetros: saludo («¡Hola, %1\$s!»), estado_en_directo, estadio y marcador, que lleva dos números. Se rellenan con getString(id, args...).
- plurals: partidos_hoy distingue 1 partido de 3 partidos. Lo añadí porque Lint lo sugería.
- Nombres propios: equipos, estadio y nombre de la app llevan translatable="false", así que no se traducen y Lint no avisa de que faltan en values-en.

Accesibilidad: las descripciones (contentDescription) también están en strings.xml

Colores

 <code>color_primario</code>	#1B5E20	Verde césped: el color que cualquiera asocia al fútbol. Se usa en la cabecera, el marcador y el icono.
 <code>color_primario_oscuro</code>	#0D3B12	Barra de estado y navegación; las costuras del balón del icono.
 <code>color_secundario</code>	#FFC107	Amarillo marcador electrónico / tarjeta: botones principales, lema y logo. Contrasta mucho con el verde.
 <code>color_en_directo</code>	#D32F2F	Rojo «EN DIRECTO»: solo para lo que está pasando ahora, para que llame la atención.
 <code>color_fondo</code>	#F3F6F1	Fondo blanco roto con un toque verde, más suave que el blanco puro.
 <code>color_superficie</code>	#FFFFFF	Fondo de las tarjetas de partido, para que destaquen sobre el fondo.
 <code>color_texto_principal</code> /  <code>color_texto_secundario</code>	#1A1A1A / #5F6B5C	Texto principal casi negro y texto secundario gris verdoso. Ambos superan un contraste de 4,5:1 sobre blanco.
 <code>color_marcaador</code> ·  <code>color_divisor</code>	#1B5E20 · #DCE3D8	Color propio del marcador (en modo oscuro cambia a un verde claro) y líneas separadoras.

Conclusion personal

Lo que más he aprendido es que, en una app de Android, ordenar bien los textos, colores e imágenes importa casi tanto como programar. Al principio me parecía una molestia guardarlo todo en carpetas aparte. Después, para traducir la app al inglés y crear el modo oscuro solo tuve que añadir esas carpetas, sin cambiar nada del código. Con la versión horizontal pasó lo mismo. También entendí mejor qué pasa cuando la app se usa de verdad: al girar el móvil, la pantalla se vuelve a crear desde cero. Y dar a los colores nombres según su función (como «color principal» en vez de «verde») fue buena idea, porque en modo oscuro el verde del marcador tenía que cambiar.



Lo más difícil:

- Configurar los emuladores, que ocupan mucho y tardan en arrancar la primera vez.
- Entender la diferencia entre dp y sp: los dos son unidades de medida, una para tamaños y otra para el texto.
- Corregir los avisos que me daba el programa, como escribir «1 partido» y «3 partidos» de forma correcta